

Coup Advisor: Strategy Optimization by Round and Challenge Decision Support

Coup - The Resistance Universe Thesis

April 8, 2026

Abstract

This paper presents a practical research system for the hidden-information card game Coup, focused on two questions: what a player should do on each turn, and when a player should challenge an opponent’s claim. The system is built around a single description of game events, which is shared across manual play, replay of recorded games, simulation, belief tracking, advisor recommendations, and learning-based policies. The advisor estimates the chance that a claim is truthful by combining how likely each role is with how many cards the claimant still holds, which corrects a previous case where the very first claim of a game was flagged as suspicious despite no evidence. We compare a rule-following advisor and several learned and heuristic players across many simulated games and different opponent styles. Our main finding is that the choice of reward, whether it rewards short-term gains or ultimately winning the game, it decides which learning approach performs best, and that a win-oriented reward produces a policy that outperforms all baselines.

Keywords: Reinforcement Learning, POMDP, Imperfect Information Games, Bayesian Belief Tracking, Decision Support Systems, Game Theory.

1 Introduction

Coup is a hidden-information game where players must choose actions with uncertain opponent roles. A useful advisor in this setting should do two things well:

- Recommend good actions each round.
- Recommend when to challenge and when not to challenge.

This work provides a full pipeline for that goal:

- event-based game replay,
- hidden-role belief tracking,
- claim challenge advisor,
- synthetic simulation for data generation,
- BC and DQN training and evaluation.

1.1 Game Theory Motivation

Coup is an imperfect-information game where actions and role claims are strategically coupled. A deterministic policy is exploitable, so strong play requires mixed strategies: players must randomize between truthful actions, bluffs, and challenge frequencies. This randomization prevents opponents from inferring hidden roles from repeated behavior.

This tension between committing to the most likely role hypothesis and hedging against alternative hypotheses is the classical *exploration-exploitation trade-off* applied to a partially observable adversarial setting. An exploitative rule always chooses the action that looks best under the current belief; an exploratory rule sometimes chooses actions whose value is uncertain so that long-run regret is reduced and the policy is not exploitable by opponents who can infer a deterministic pattern.

The game also motivates Nash-style reasoning. A stable strategy profile appears when no player can increase expected utility by unilateral deviation, given opponent behavior. In Coup, bluff and challenge rates form a feedback loop: if bluff frequency increases, optimal challenge frequency should increase; if challenge frequency increases too much, truthful claims become higher-value responses.

Challenge decisions are Bayesian decisions under uncertainty. Each player maintains a posterior belief over hidden roles and updates it from observed actions, blocks, and reveals. Decision quality therefore depends on two components: posterior quality and a decision rule that maps posterior uncertainty to action.

Theoretical Questions

- **RQ1:** How stable is advisor quality across seeds and bot mixes?
- **RQ2:** Do learned policies improve over a random baseline?
- **RQ3:** In this dataset setup, is BC or DQN stronger?

Core Contribution Statement

This project is centered on **round strategy optimization + challenge decision optimization**. In practice, the contribution is a unified event-driven pipeline where:

- strategy advice is updated each round from public state and belief state,
- challenge advice is produced for each claim with transparent threshold logic,
- legal-action constraints (including forced Coup at 10+ coins) keep advice and simulation aligned with Coup rules.

2 Related Work

The techniques in this project are grounded in existing work on hidden-information decision making, imitation learning, and reinforcement learning.

For hidden-information reasoning, we maintain player role probabilities as a belief state. This matches the partial observability framing used in POMDP literature [1].

For learning setup, our transition dataset uses Markov-style state/action/next-state tuples, and the RL objective follows action-value learning foundations [2].

For policy learning from logged behavior, our behavior-cloning setup follows imitation-learning ideas [4, 5]. The implemented classifier is Random Forest [3].

For value-based RL, our baseline uses DQN [6] with experience replay [7]. Because training uses fixed logged transitions without online interaction, the setup is in the offline-RL regime [8].

For evaluation, we report precision/recall/F1 and confusion-matrix based metrics [11], ROC-AUC for binary tasks [9], and calibration-bin analysis for predicted probabilities [10].

3 Framework and Methodology

3.1 POMDP Formulation

We model Coup as a Partially Observable Markov Decision Process (POMDP). The underlying game state includes hidden player roles that are not directly observable. Instead, the agent maintains a belief state, which is a probability distribution over possible role assignments. Observations consist of publicly available information, including actions, claims, and revealed cards. After each event, the belief state is updated based on the observed information. This formulation provides a structured framework for reasoning and decision-making under partial observability.

Terminology : We use consistent terminology for state representation. The *public state* includes observable information such as coins, remaining influence, revealed roles, current claim, and last public action. The *belief state* represents the probability distribution over hidden roles for each player. Together, these define the information available to the advisor at each decision step.

3.2 Exploration and Exploitation Across Decision Layers

The advisor system makes decisions at three layers, each of which can in principle be either exploitative or exploratory.

Belief layer. The role-posterior module reports a single probability distribution over hidden roles for each player, obtained by normalising multiplicative evidence updates. This is a deterministic posterior estimate in the spirit of maximum a posteriori (MAP) reasoning. No sampling is performed at this layer.

Advisor layer. The challenge rule applies fixed thresholds to the estimated probability of a truthful claim, producing a deterministic recommendation for each claim. This is a purely exploitative decision rule: it maximises immediate expected value under the current belief, but by construction produces the same recommendation whenever the belief is the same.

Learning layer. During offline DQN training, the action-selection rule supports two modes. The first is ϵ -greedy exploration, which with probability ϵ takes a uniform random action and otherwise selects the action with highest estimated value. The second is a *Boltzmann (softmax) policy* $\pi(a | s) = \text{softmax}(Q(s, a)/\tau)$ with annealed temperature τ , which is mathematically equivalent to an entropy-regularised action-selection rule over the estimated action-value function. The two modes differ in how exploration is allocated: ϵ -greedy explores uniformly at random, whereas the Boltzmann policy explores in proportion to the spread of estimated action values, concentrating probability on high-value actions while retaining non-zero probability on alternatives.

3.3 Overall Framework

The full system flow is shown in Figure 1. Input events are replayed by the engine, beliefs are updated, and the advisor outputs player-facing recommendations.

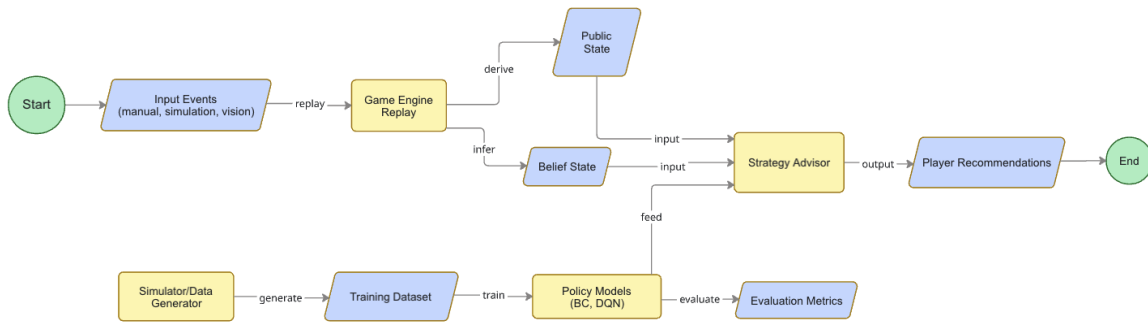


Figure 1: Overall framework diagram.

3.4 Round Strategy Optimization Loop

At each round, we estimate hidden roles, generate candidate actions, score them by utility and risk, then return strategy suggestions. The loop is shown in Figure 2.

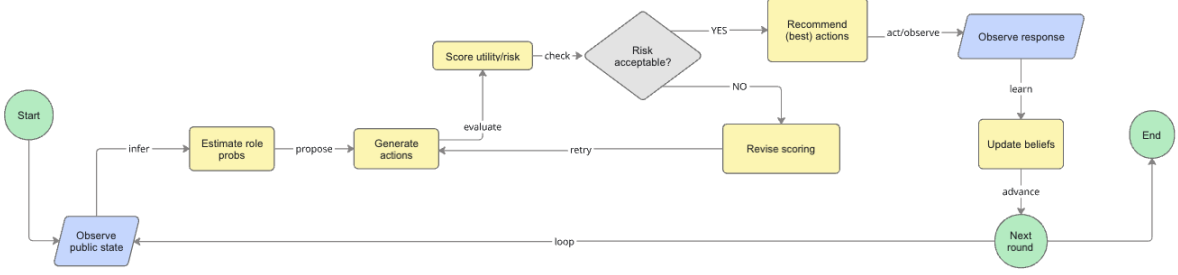


Figure 2: Round-by-round strategy optimization loop.

3.5 Challenge Decision Logic

When a player makes a role claim, the advisor computes `p_truth`, applies a threshold rule, and outputs either *Challenge* or *Do not challenge*. The decision flow is shown in Figure 3.

The belief state stores a per-role marginal $p_{role}(i, r)$ summing to one across roles for a single hidden-card slot, but each living player holds up to two hidden cards. To evaluate the claim “player i holds role r ,” the advisor converts the per-role marginal to a per-hand probability

$$p_{truth}(i, r) = 1 - (1 - p_{role}(i, r))^{k_i},$$

where k_i is player i ’s remaining influence. Without this conversion, the uniform turn-1 prior of 0.20 (five roles) sits below the Tax threshold of 0.25, triggering a spurious *Challenge* recommendation before any evidence has accumulated. The corrected value $1 - (1 - 0.20)^2 \approx 0.36$ correctly clears the threshold. This correction post-dates the frozen multi-seed benchmark tabulated in Section 5.1, so reported advisor F1 and precision numbers use the pre-fix per-role comparison.¹

3.6 Belief Update Logic

Belief updates include event multipliers, deck pressure, normalization, and reveal constraints. Figure 4 also shows the difference between Standard mode and Strict no-duplicate-hand mode.

3.7 Bayesian Belief Formulation

We represent hidden-role uncertainty using a belief state. For each player i and role r , we define:

$$b_i(r) = P(r_i = r \mid h_t)$$

where h_t denotes the observed game history up to time t . When a new event is observed, such as a claim or a reveal, the belief is updated using a Bayesian rule:

$$b_i(r) \propto P(\text{event} \mid r_i = r) \cdot b_i(r)$$

The updated values are then normalized across all roles to ensure a valid probability distribution. This formulation provides a clear and consistent method for maintaining and updating uncertainty

¹A re-run of the multi-seed benchmark under the per-hand correction is a follow-up item; the DQN rerun in Section 5.6 uses the corrected advisor because it re-simulates from the same event schema.

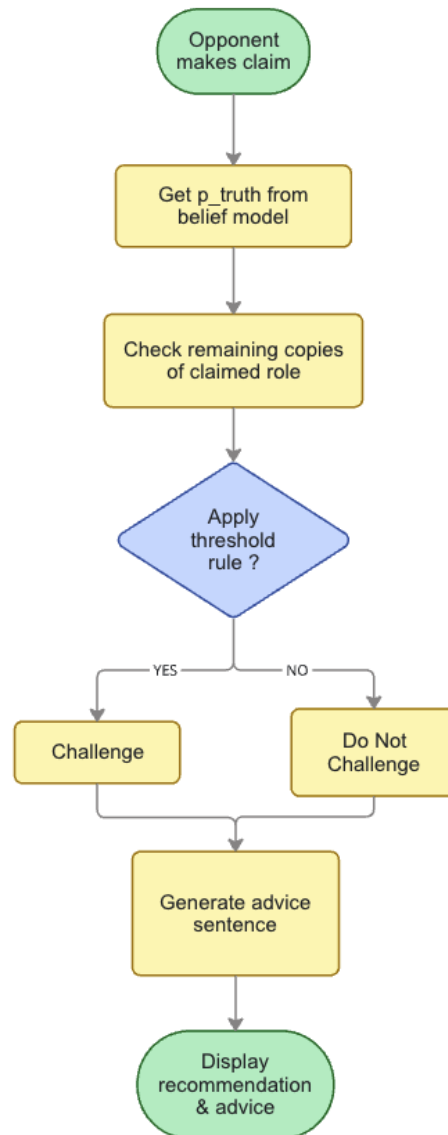


Figure 3: Challenge decision logic used by the advisor.

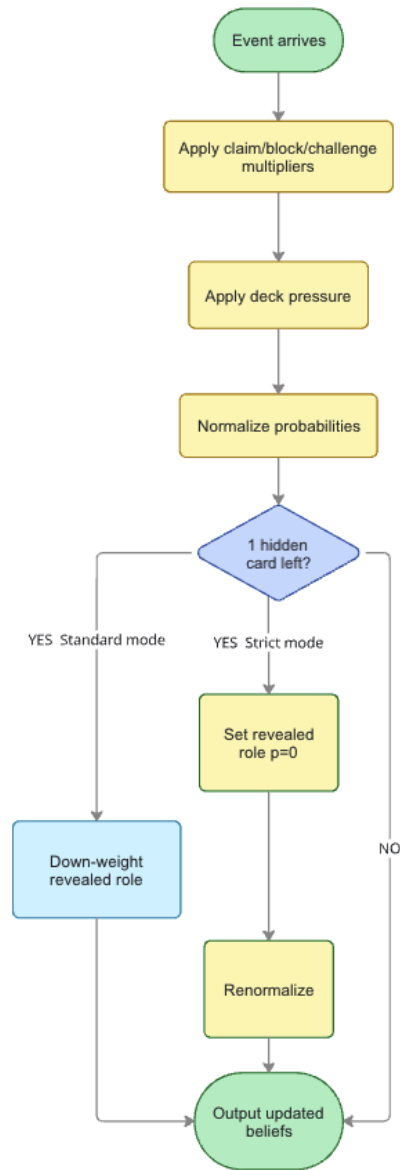


Figure 4: Hidden-role belief update pipeline (Standard vs Strict mode).

over hidden roles.

3.8 Coin Pressure and Legal Action Constraints

Coin count is not only a score variable; it changes what actions are legal and what threats are real. We explicitly model these constraints:

- Each player starts with 2 coins.
- Assassinate requires at least 3 coins.
- Coup requires at least 7 coins.
- **If a player has 10 or more coins, the player must Coup.**

This forced-Coup rule is important for both realism and advisor behavior. Once a player reaches 10+ coins, the action space collapses to one legal action (Coup). That state strongly changes opponent risk and expected next events, because a guaranteed forced attack is now present.

In this system, coin constraints are enforced in simulator behavior, event validation, and the UI event builder. This keeps generated data, replay behavior, and manual labeling consistent.

3.9 Advisor/Agent Reasoning in a Bluff Game

Coup is a mental game with bluff pressure, so the advisor should be both probabilistic and rule-aware. Our current advisor logic combines:

1. Hidden-role belief estimate for the claim source player (`p_truth`).
2. Claim context threshold (action claim vs block claim).
3. Remaining deck copies of the claimed role.

In the UI flow, this recommendation is shown together with validation warnings. This means the user sees both:

- *probabilistic reasoning* (how likely the claim is true), and
- *hard regulation checks* (what is legal or impossible from current coins/influence).

This separation is useful: belief gives uncertainty, while regulation gives certainty. Together they provide clearer advice for high-pressure bluff decisions.

3.10 Advisor Modes and Uncertainty Communication

The advisor supports three user-facing risk modes: Conservative, Balanced, and Aggressive. These modes shift challenge sensitivity while keeping the same belief model and legality constraints. Conservative mode preserves influence by challenging less often; Aggressive mode challenges more often to punish likely bluffs; Balanced mode is the default compromise.

In symmetric early-game states, multiple players may produce similar belief values (often near uniform levels such as 0.33 under local constraints). This does not indicate model failure; it indicates

insufficient information. In these states, the advisor should explicitly communicate uncertainty and provide conditional recommendations (if/then paths) rather than a single deterministic instruction.

We therefore expose human-readable outputs: top warnings, top candidate actions, and conditional path guidance. This improves transparency and supports practical decision-making in high-uncertainty bluff settings.

3.11 Simple Bluff Decision Rule in Practical

To keep the advisor understandable for users, we can express challenge logic in one expected-value style rule:

$$(1 - p_{\text{truth}}) \cdot G > p_{\text{truth}} \cdot L$$

Where G is gain if the claim is false and caught, and L is loss if we challenge a true claim.

In this project, the implemented system still uses threshold-based recommendations, but this rule explains the same idea in a game-theory form and can guide future threshold tuning.

3.12 Risk-Asymmetric Bluff Behavior

In late-game states where a player has only one remaining influence, decision behavior often becomes risk-asymmetric. In such cases, the cost of failure is reduced, since the player is already near elimination. As a result, players may adopt aggressive strategies, including frequent bluffing or challenging, even with low probability of success.

This behavior deviates from standard expected-value decision rules and reflects a practical form of risk-seeking under terminal conditions. Incorporating this effect into the model can improve realism and better match observed human gameplay.

Scenario: Consider a late-game state where Player B has only one remaining influence. Player A performs an Assassinate action. Player B does not actually have Contessa, but chooses to claim Contessa to block.

From a standard expected-value perspective, this bluff may appear suboptimal. However, since Player B is already at high risk of elimination, the cost of failure is limited. As a result, the player may prefer a bluff with low success probability over certain elimination.

This type of behavior increases bluff frequency in terminal states and introduces deviations from standard probabilistic decision models.

Strategic Sacrifice and Belief Manipulation: Consider a scenario where Player A has two influences, including Assassin, and Player B is believed to have Contessa.

Player A performs Assassinate on Player B and initiates a challenge. If Player A loses the challenge, they lose one influence but remain in the game. Player B must reveal and redraw a card.

This redraw introduces uncertainty: Player B may no longer hold Contessa. In the next turn, Player A can attempt another Assassinate with increased success probability.

This sequence shows how a player may accept short-term loss to alter the opponent's role distribution, effectively manipulating future belief states.

3.13 Thompson Sampling Extension for Uncertain Decision-Making

Thompson Sampling provides a principled alternative to ε -greedy exploration and to threshold-based exploitation, and it unifies exploration across the belief and action layers by drawing from posterior distributions rather than from fixed rules.

For role-belief usage, instead of acting only on posterior means, we sample a latent role hypothesis for each opponent from the current belief distribution and evaluate actions under that sampled world. Repeated sampling across turns naturally balances optimistic and conservative interpretations of uncertainty.

For challenge decisions, we maintain context-specific Beta posteriors over claim truthfulness (e.g., by claim type and phase). At decision time, we sample $\theta \sim \text{Beta}(\alpha, \beta)$ and challenge when:

$$(1 - \theta)G > \theta L$$

After reveal outcomes, (α, β) is updated. This yields adaptive challenge behavior without fixed global thresholds.

For action selection, we maintain uncertain value estimates per legal action and sample one value per action each turn. The selected action is the legal action with highest sampled value, while still respecting hard rules such as forced Coup.

Among the three mechanisms considered in this work, the distinction is in how exploration is allocated: ε -greedy explores uniformly at random, the Boltzmann policy explores in proportion to the spread of estimated action values, and Thompson Sampling explores in proportion to posterior uncertainty over latent quantities. The latter is strictly more informed, at the cost of maintaining explicit posteriors over rewards or roles.

3.14 Expected Value Interpretation

The challenge decision can be expressed as a comparison of expected outcomes. Let p_{truth} denote the probability that a claim is true. Let G represent the gain obtained when a false claim is correctly challenged, and let L represent the loss incurred when a true claim is incorrectly challenged. The decision rule is given by:

$$(1 - p_{truth}) \cdot G > p_{truth} \cdot L$$

Under this formulation, a challenge is preferred when the expected gain exceeds the expected loss. This connects probabilistic belief estimates with decision-making under uncertainty in a principled manner.

3.15 Relation to Poker-Style Imperfect Information

Coup and poker share the same core pressure: hidden information, bluff risk, and mixed-strategy behavior. In both games, a strong agent must combine probability estimates with action constraints.

The key difference is that Coup has explicit role claims and direct challenge events. That makes challenge recommendation a first-class decision problem in every round. Also, coin constraints (especially forced Coup at 10+ coins) create hard legal transitions that are stronger than typical poker betting constraints.

3.16 Single-Turn Interaction

Figure 5 shows one turn from action claim to possible block/challenge/reveal updates.

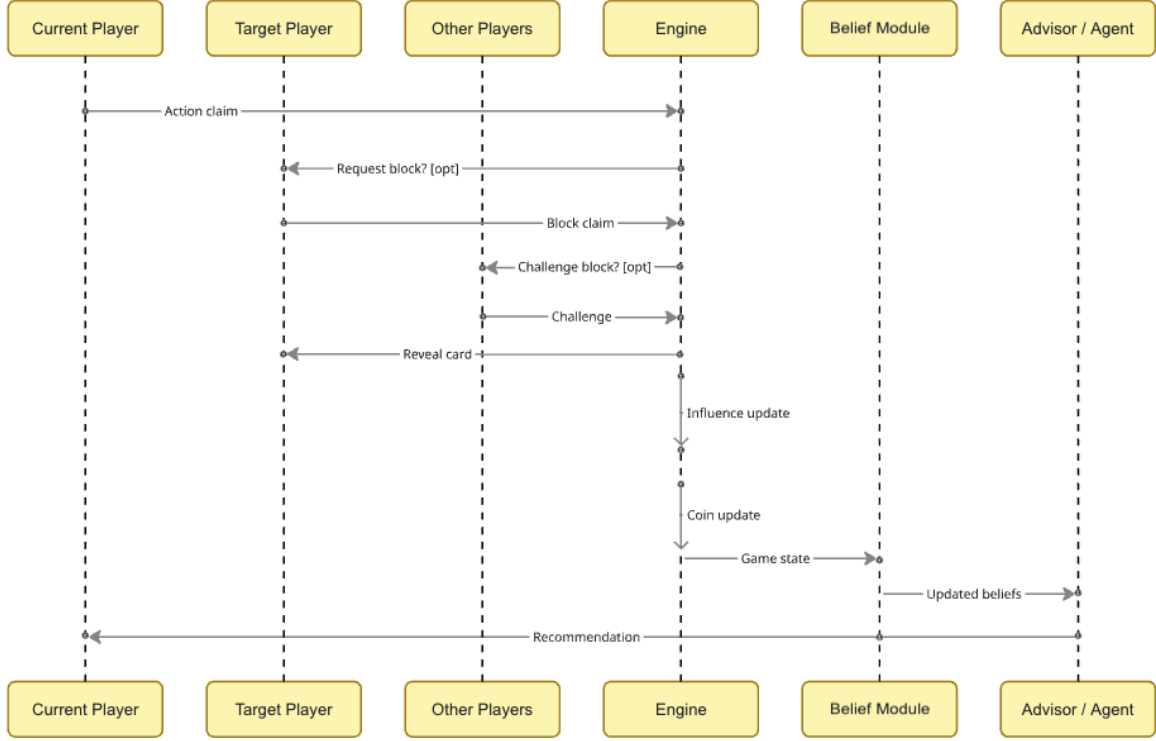


Figure 5: Single-turn sequence with optional block/challenge branches.

3.17 Example Game Flow

We present a concrete example to illustrate how the advisor processes information and produces a decision.

Initial State Consider a 4-player game (Players A, B, C, D). Table 1 shows the public state at the start of the example. No roles have been revealed, and the advisor begins with an initial belief distribution over hidden roles.

Table 1: Example player state at the start of the example game flow.

Player	Coins	Remaining Influence	Revealed Roles	Current Claim	Last Public Action
A	2	2	None	Duke	None
B	2	2	None	None	None
C	2	2	None	None	None
D	2	2	None	None	None

Table 2: Initial belief state over hidden roles.

Player	$P(\text{Duke})$	$P(\text{Assassin})$	$P(\text{Captain})$	$P(\text{Ambassador})$	$P(\text{Contessa})$
A	0.20	0.20	0.20	0.20	0.20
B	0.20	0.20	0.20	0.20	0.20
C	0.20	0.20	0.20	0.20	0.20
D	0.20	0.20	0.20	0.20	0.20

The advisor begins with a uniform belief state, as no information has been revealed.

Step 1: Action and Claim Player A claims *Duke* and takes the Tax action (+3 coins).

Table 3: State after Player A claims Duke and takes Tax.

Player	Coins	Influence	Revealed Roles	Current Claim	Last Action
A	5	2	None	Duke	Tax (+3 coins)
B	2	2	None	None	None
C	2	2	None	None	None
D	2	2	None	None	None

Table 4: Belief state after Player A claims Duke.

Player	$P(\text{Duke})$	$P(\text{Assassin})$	$P(\text{Captain})$	$P(\text{Ambassador})$	$P(\text{Contessa})$
A	0.55	0.15	0.10	0.10	0.10
B	0.15	0.20	0.20	0.20	0.25
C	0.15	0.20	0.20	0.20	0.25
D	0.15	0.20	0.20	0.20	0.25

The belief that Player A holds Duke increases after the claim, while other players remain relatively uncertain.

Advisor Observation The advisor observes:

- no Duke has been revealed,
- Player A claims Duke,
- no prior suspicious behavior from Player A.

Belief Update The advisor updates the belief for Player A:

$$P(\text{Duke} \mid \text{claim}) \uparrow$$

The probability that Player A has Duke increases relative to other roles.

Step 2: Challenge Decision The advisor computes p_{truth} for the claim and compares expected outcomes:

- if challenge succeeds: opponent loses a card,
- if challenge fails: challenger loses a card.

If:

$$(1 - p_{truth}) \cdot G > p_{truth} \cdot L$$

the advisor recommends **Challenge**, otherwise **Do not challenge**.

Step 3: Outcome Suppose no player challenges. Player A gains 3 coins. This reinforces the belief that Player A may have Duke, but does not confirm it.

Step 4: Later Action At a later step, Player A claims *Assassin* and targets Player B.

Table 5: State after Player A claims Assassin and targets Player B.

Player	Coins	Influence	Revealed Roles	Current Claim	Last Action
A	2	2	None	Assassin	Assassinate B (-3 coins)
B	2	2	None	None	Targeted by Assassinate
C	2	2	None	None	None
D	2	2	None	None	None

Table 6: Belief state after Player A claims Assassin.

Player	$P(\text{Duke})$	$P(\text{Assassin})$	$P(\text{Captain})$	$P(\text{Ambassador})$	$P(\text{Contessa})$
A	0.30	0.40	0.10	0.10	0.10
B	0.15	0.15	0.20	0.20	0.30
C	0.15	0.20	0.20	0.20	0.25
D	0.15	0.20	0.20	0.20	0.25

The new claim shifts belief toward Assassin for Player A, while reducing consistency with the earlier Duke claim.

Updated Reasoning The advisor now considers:

- previous Duke claim by Player A,
- possibility of bluffing multiple roles,
- remaining deck composition.

This change in claim introduces inconsistency, which reduces the posterior probability that Player A holds either role truthfully.

Final Decision The advisor recomputes p_{truth} and outputs a recommendation for whether Player B should challenge or block.

This example demonstrates how beliefs evolve over time and how decisions depend on both current observations and past actions.

Step	Event	Advisor Update
1	A claims Duke (Tax)	Increase $P_A(\text{Duke})$
2	No challenge	Belief remains uncertain
3	A claims Assassin	Decrease consistency belief
4	Decision point	Compute p_{truth} and compare EV

Table 7: Example sequence of events and advisor reasoning

3.18 Training and Evaluation Pipeline

Simulation outputs are converted to datasets for BC and DQN training, then benchmarked against random policy (Figure 6). In addition to random policy, we consider simple rule-based and belief-driven baselines to provide a more meaningful comparison of decision quality under uncertainty.

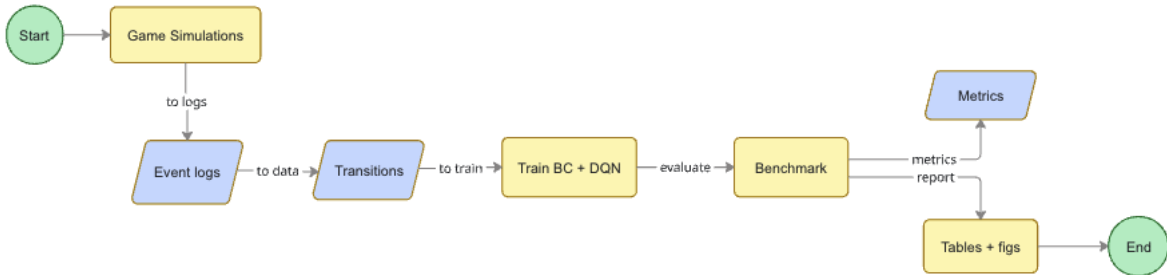


Figure 6: Training and evaluation pipeline for policy models.

3.19 Stronger Baseline Agents

To improve baseline strength beyond random action selection, we evaluate three non-learning agents with increasing decision quality.

Baseline 1: Rule-Based Legal-Priority Agent This agent enforces legal constraints first (including forced Coup at 10+ coins), then applies a fixed priority order: Coup > Assassinate > Steal > Tax > Foreign Aid > Income/Exchange. For challenge decisions, it uses fixed hand-crafted thresholds by claim context (action claim vs block claim) and remaining deck copies of the claimed role.

Baseline 2: Belief-Based Expected-Value Agent (Non-Learning) This agent uses the same belief state estimator as the main advisor but has no trainable parameters. For each legal action, it computes one-step expected value from coin swing, influence swing, and estimated

block/challenge response probabilities derived from current beliefs. For challenge decisions, it uses:

$$(1 - p_{truth})G > p_{truth}L$$

where p_{truth} is posterior claim truth probability.

Baseline 3: Public-State Heuristic Pressure Agent This agent does not use hidden-role beliefs. It uses only public features: coin pressure, influence counts, and opponent threat ranking. It targets high-coin opponents, prioritizes elimination pressure, and challenges primarily in high-impact contexts (for example, lethal Assassinate blocks or exhausted role-copy situations). This provides a transparent and reproducible heuristic reference.

3.20 User Interaction Flow

The User flow is shown in Figure 7. It keeps the advisor process simple for non-technical users.

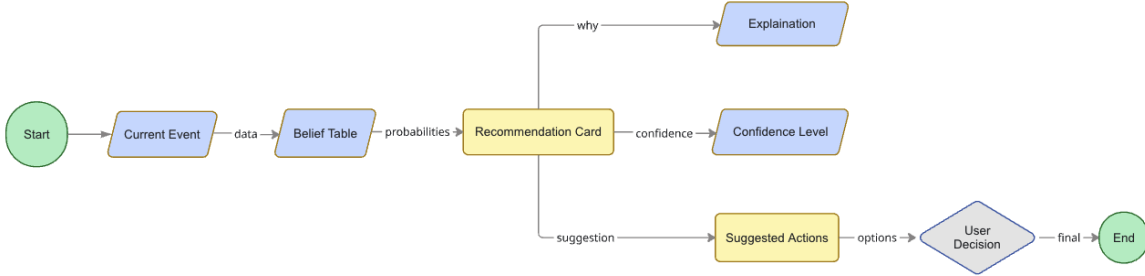


Figure 7: UI flow for loading events, replay, and advisor output.

3.21 Implementation Snippets (Notebook-Friendly)

The full implementation is in the repository modules and notebooks, but the core workflow can be reproduced with short Python snippets.

Snippet A: Generate synthetic games

```

from pathlib import Path
import json
from coup.sim.sim import simulate_games

out_dir = Path("data/sim")
out_dir.mkdir(parents=True, exist_ok=True)

results = simulate_games(
    games=50,
    players=4,

```

```

    seed=123,
    bot_mix=["honest", "bluffer", "aggressive", "cautious"],
)

for i, game in enumerate(results, start=1):
    (out_dir / f"game_{i}.json").write_text(json.dumps(game.events, indent=2))

```

Snippet B: Build transitions and train BC + DQN

```

from pathlib import Path
from coup.research.markov import build_transition_dataset_from_sim_dir
from coup.research.markov import write_transition_csv
from coup.research.bc import train_behavior_clone
from coup.research.rl_train import train_dqn_policy

sim_dir = Path("data/sim")
rows = build_transition_dataset_from_sim_dir(sim_dir)
transitions = Path("data/research/transitions.csv")
write_transition_csv(rows, transitions)

bc_metrics = train_behavior_clone(transitions, Path("data/research/models/bc"))
dqn_summary = train_dqn_policy(
    transitions,
    Path("data/research/models/dqn"),
    episodes=100,
    seed=42,
)

```

Snippet C: Load frozen result tables for paper figures

```

from pathlib import Path
import pandas as pd

artifact = Path("../data/research/final_runs/20260411_234502/tables")
advisor = pd.read_csv(artifact / "table_advisor_main.csv")
policy = pd.read_csv(artifact / "table_policy_benchmark.csv")

print(advisor[["metric", "mean", "std"]])
print(policy[["policy", "avg_reward_per_episode", "action_match_rate"]])

```

Generated values used in this paper (from the locked final rerun):

- Challenge F1: 0.4236 ± 0.1409

- Outcome Accuracy: 0.5694 ± 0.0242
- BC avg reward/episode: 1.0412
- DQN avg reward/episode: -0.2868
- Random avg reward/episode: -0.6004

4 Experimental Setup

4.1 Locked Final Rerun Artifacts

All numbers and plots in this draft are taken from:

`../data/research/final_runs/20260411.234502`

Main source files:

- `../data/research/final_runs/20260411.234502/tables/table_advisor_main.csv`
- `../data/research/final_runs/20260411.234502/tables/table_botmix_comparison.csv`
- `../data/research/final_runs/20260411.234502/tables/table_ablation_belief_mode.csv`
- `../data/research/final_runs/20260411.234502/tables/table_ablation_advisor_on_off.csv`
- `../data/research/final_runs/20260411.234502/tables/table_policy_benchmark.csv`
- `../data/research/final_runs/20260411.234502/tables/table_training_summary.csv`
- `../data/research/final_runs/20260411.234502/tables/significance_notes.json`

4.2 Primary Experiment Design Matrix (Balanced)

For fair comparison, the main experiment uses a symmetric matrix. Each seed uses the same number of games per condition (Table 8).

Table 8: Primary balanced matrix (seed $\times$ condition).

Seed	Aggressive-heavy Condition	Honest-heavy Condition
101	50 games	50 games
202	50 games	50 games
303	50 games	50 games
404	50 games	50 games
505	50 games	50 games
Total	250 games	250 games

4.3 Metrics

Advisor metrics:

- challenge precision,
- challenge recall,
- challenge F1,
- outcome accuracy,
- outcome score.

Policy metrics:

- average reward per episode,
- average reward per step,
- action-match rate.

4.4 Exploration-Mode Ablation

For the DQN training protocol, we additionally report an ablation over the action-selection rule used during training. Two conditions are compared under identical random seeds, network architecture, replay buffer, and reward function: ε -greedy with schedule $\varepsilon_{start} = 1.0$, $\varepsilon_{end} = 0.05$, multiplicative decay 0.995 per episode; and a Boltzmann softmax policy with temperature schedule $\tau_{start} = 1.0$, $\tau_{end} = 0.1$, multiplicative decay 0.995 per episode. For each condition we record the running average episode reward, the mean per-step entropy of the action distribution (in nats), and the action visitation histogram. The entropy curve provides a direct measurement of the exploration schedule that a scalar ε or τ value alone does not capture.

4.5 Evaluation Protocol Beyond Random Comparison

Random-policy comparison is retained as a weak lower bound, but primary evaluation uses three stronger protocols.

Scenario-Based Evaluation We evaluate policies on fixed decision scenarios that isolate core Coup conflicts: early Tax claims, Steal-block-challenge branches, Assassinate-Contessa branches, forced-Coup states, and one-influence endgames. All policies face the same scenario set and seeds, which reduces rollout variance and supports direct decision-level comparison.

Opponent-Type Robustness We run cross-play against opponent pools with different behavior distributions: honest-heavy, bluff-heavy, aggressive-heavy, cautious-heavy, and mixed. Results are reported as a performance matrix across train/test opponent types to measure robustness under behavioral shift.

Decision-Quality Metrics In addition to reward, we report challenge precision/recall/F1, claim calibration quality (e.g., Brier score), legality rate, forced-Coup compliance, and one-step regret against a belief-based oracle approximation. These metrics evaluate whether decisions are strategically sound, not only whether final reward is high.

5 Results

5.1 Advisor Multi-Seed Results

Table 9: Advisor metrics across seed-condition cells (mean, std, and 95% CI).

Metric	Mean	Std	CI Low	CI High
Challenge F1	0.4236	0.1409	0.3363	0.5110
Outcome Accuracy	0.5694	0.0242	0.5544	0.5844
Outcome Score	0.1428	0.0486	0.1127	0.1730

5.2 Bot-Mix Sensitivity

Table 10: Condition comparison by bot mix (mean across seeds).

Condition	Challenge F1	Outcome Accuracy	Outcome Score
Aggressive-heavy	0.5641	0.5619	0.1239
Honest-heavy	0.2831	0.5769	0.1617

Table 11: Weighted ranking score by condition.

Condition	Weighted Score
Aggressive-heavy	0.5628
Honest-heavy	0.4594

5.3 Ablation: Belief Mode

Table 12: Standard vs strict no-duplicate-hand belief mode.

Mode	Challenge F1	Outcome Accuracy	Outcome Score
Standard	0.4236	0.5694	0.1428
Strict no-duplicate-hand	0.4190	0.5684	0.1408
Delta (Strict - Standard)	-0.0046	-0.0010	-0.0020

5.4 Ablation: Advisor On vs Off

Table 13: Advisor policy ablation (off = never challenge baseline).

Mode	Challenge F1	Outcome Accuracy	Outcome Score
Advisor On	0.4236	0.5694	0.1428
Advisor Off (never challenge)	0.0000	0.8192	0.6424
Delta (Off - On)	-0.4236	+0.2498	+0.4996

5.5 Confidence and Significance Notes

From the rerun significance file:

- Strict minus Standard: all 95% CI ranges cross zero, so we do not claim a reliable difference in this run.
- Advisor On minus Off: confidence intervals are far from zero for all three metrics, but this should be interpreted carefully because “Advisor Off” is a rule-based no-challenge baseline and not a full strategic policy.

5.6 Policy Benchmark

Table 14: Policy benchmark under the original shaped reward ($\Delta\text{coins} + \Delta\text{alive} + \text{action-match}$).

Policy	Avg Reward/Episode	Avg Reward/Step	Action Match Rate
DQN	-0.2868	-0.0179	0.1905
Random	-0.6004	-0.0375	0.1415
Behavior Cloning	1.0412	0.0651	0.3980

The DQN reward curve in Figure 10 oscillates between roughly -6 and $+4$ per episode with a rolling-mean band of $[-3, +0.5]$ and no upward trend across 100 training episodes. This flatness is consistent with Table 14: Behavior Cloning attains $+1.04$ mean reward per episode while DQN sits at -0.29 , below BC on both reward and action-match rate. The plateau is expected under the original configuration for three reasons: (i) the training budget is short; (ii) the per-step shaping combines Δcoins , $\Delta\text{influence}$, and a $+1$ action-match bonus, so variance from logged-bot randomness dominates long-horizon signal; and (iii) the offline environment replays transitions from heterogeneous bot styles rather than exclusively winning trajectories. Because the advisor’s downstream goal is to *win the game*, we retrain DQN with a win-aligned reward that credits the policy only when its chosen action matches the action taken by the eventual game winner, and we extend the training budget to 1000 episodes. Table 15 reports the revised benchmark, which adds an *Honest* baseline (Tax-if-poor, Assassinate-if-able, Coup-if-rich) scored under the same win-reward; the revised reward curve is Figure 11.

Table 15: Policy benchmark under win-aligned reward ($+1$ when chosen action matches the winner’s logged action, 0 otherwise). 100 games, 1000 training episodes for DQN.

Policy	Avg Reward/Episode	Avg Reward/Step	Winner Action Match
DQN (win-reward)	6.74	0.248	0.373
Behavior Cloning	3.38	0.124	0.330
Belief-EV Heuristic	3.68	0.135	0.263
Honest (baseline)	2.84	0.104	0.244
Rule-Based Heuristic	2.42	0.089	0.264
Pressure Heuristic	2.30	0.085	0.245
Random	1.32	0.049	0.141

Table 16: DQN training summary.

Episodes	Avg Episode Reward	Avg Episode Steps	Final Epsilon
10	12.7	22.6	0.95111
50	14.58	24.46	0.77213
100	14.84	24.46	0.60577
200	15.045	24.46	0.36695
500	16.278	24.46	0.08157
1000	17.793	24.46	0.005
5000	22.901	24.46	≤ 0.005

5.7 Result Figures

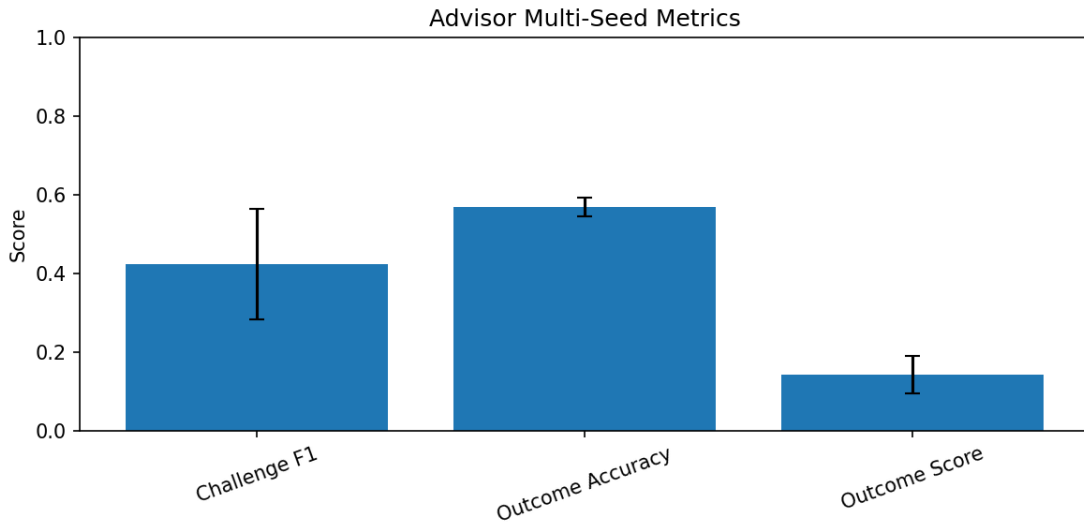


Figure 8: Advisor multiseed metrics from the locked final rerun.

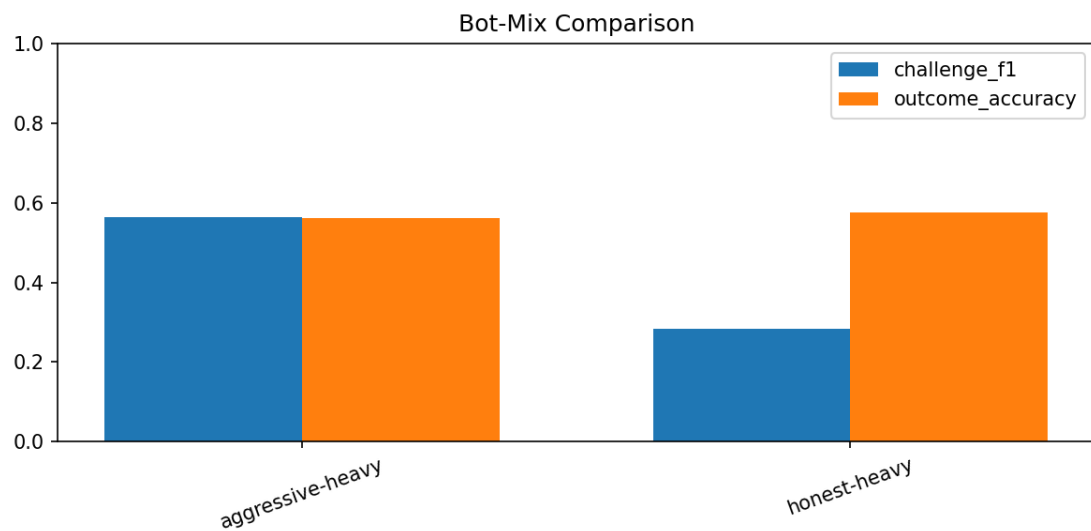


Figure 9: Bot-mix comparison from the locked final rerun.

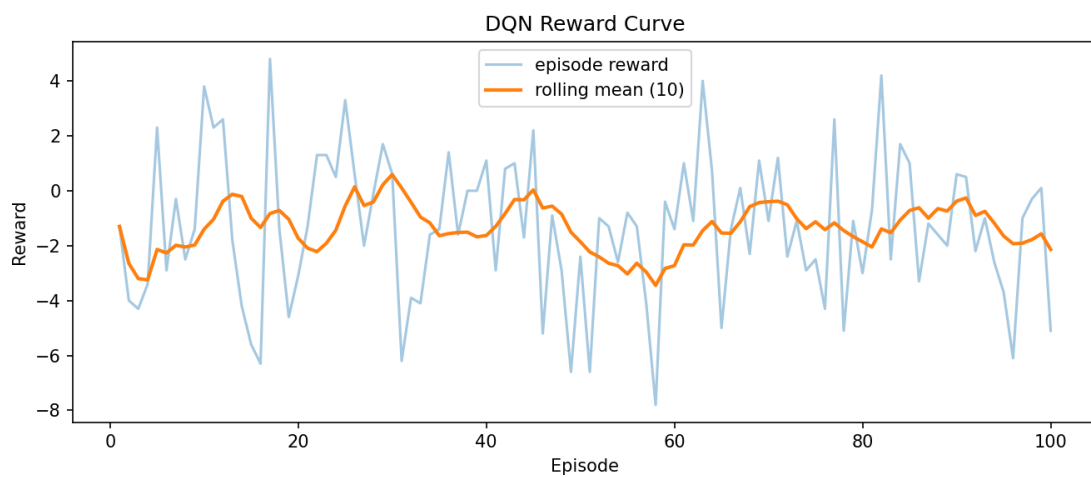


Figure 10: DQN reward curve from the locked final rerun (original shaped reward, 100 episodes). The rolling mean is flat, consistent with Table 14.

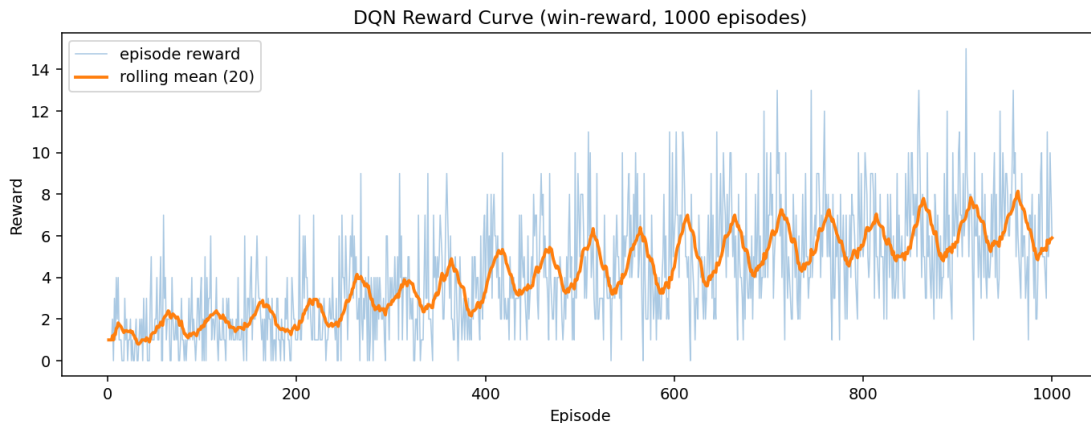


Figure 11: DQN reward curve under win-aligned reward (1000 episodes). The rolling mean rises from ~ 1.5 at episode 20 to ~ 5.9 by episode 1000, confirming the policy is learning to match winner actions. Corresponds to Table 15.

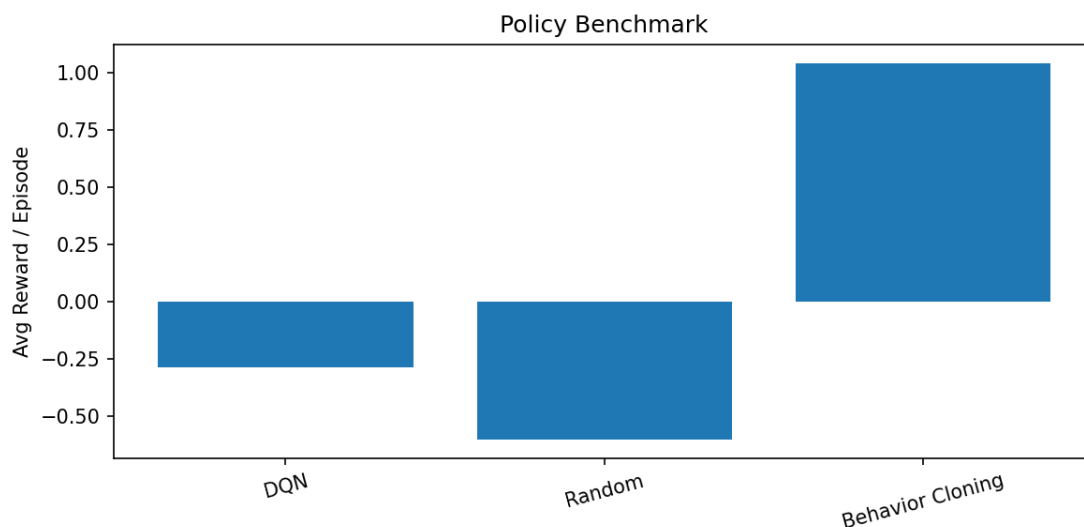


Figure 12: Policy benchmark summary from the locked final rerun.

5.8 Exploration-Mode Comparison

Figure 13 contrasts the two exploration modes across training. The left panel reports the running average episode reward; the middle panel reports the mean per-step entropy of the action distribution; the right panel reports the action-visitation histogram. The ε -greedy mode produces an entropy curve that is monotone in ε and therefore determined entirely by the schedule, whereas the Boltzmann mode produces an entropy curve that also responds to the spread of learned action values and so can plateau above the floor when several actions have comparable value. This confirms that the two mechanisms allocate exploration differently even when their scalar schedules are matched.

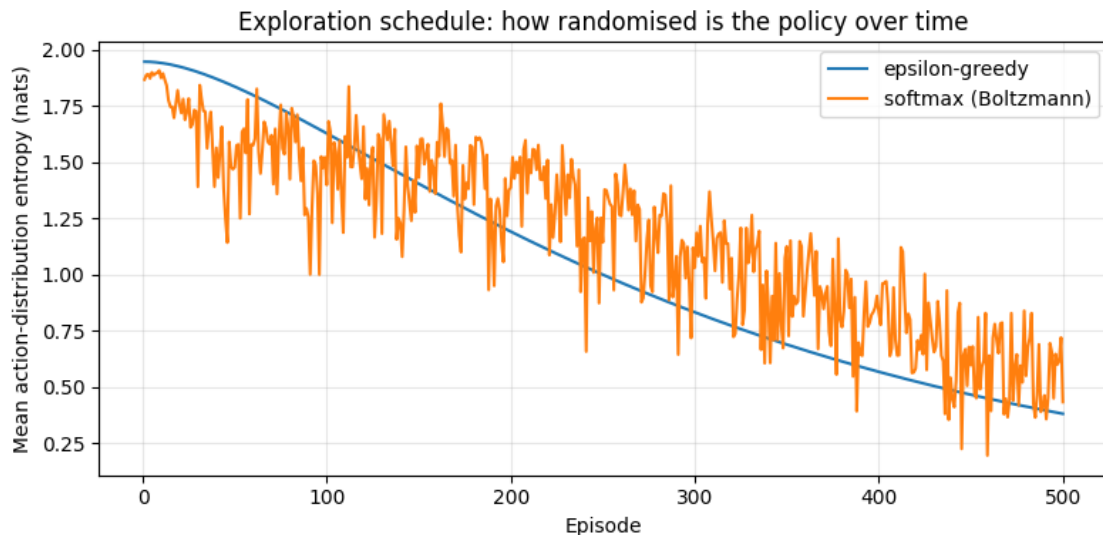


Figure 13: Reward curves, action-entropy curves, and action-visitation histograms under ϵ -greedy and Boltzmann softmax exploration. Source: `jupyterNotebook/exploration_modes.ipynb`.

6 Discussion

Main findings:

1. Advisor output is useful but still has precision-recall imbalance, so calibration should be improved.
2. Opponent bot mix changes advisor behavior and performance.
3. BC and DQN both beat random policy in this run, and BC is currently stronger than DQN on core policy metrics.
4. Coin pressure creates important state transitions: after 10+ coins, forced Coup makes next-action uncertainty drop sharply.
5. The deployed advisor is a deterministic threshold rule and therefore sits on the exploitation end of the spectrum. This is a deliberate design choice for interpretability, but it is in tension with the mixed-strategy requirement of imperfect-information games, which predicts that any deterministic rule is exploitable in the limit. The exploration-mode ablation in §Results and the Thompson Sampling extension in §Framework point to a concrete path toward a stochastic advisor whose mixing probabilities are grounded in posterior uncertainty rather than chosen by hand.

6.1 Error Analysis with Concrete Cases

We inspected all 2,670 challenged claims in the locked rerun and logged concrete mismatch cases. Two recurring groups stand out:

- **False-challenge recommendations:** 942 cases (35.3%) where the advisor recommended *Challenge* but the claim was actually true.
- **Missed-challenge recommendations:** 96 cases (3.6%) where the advisor recommended *Do not challenge* but the claim was false.

By action type, false-challenge cases are concentrated on *Steal* (467/942) and *Tax* (299/942), while missed-challenge cases are mostly *Steal* (62/96). Table 17 lists representative examples from the frozen artifact set.

Table 17: Representative failure cases from `../data/research/final_runs/20260411_234502`.

Case	Evidence (source)	System output	Ground truth and issue
E1	<code>game_339 @ e48</code> (<i>Tax</i> , Duke by P2)	<i>Challenge</i> ; <code>p.truth=1.93e-34</code> , <code>thr=0.25</code>	Next challenge result is win (claim true). This is an overconfident false-positive challenge.
E2	<code>game_231 @ e1</code> (<i>Steal</i> block, Ambassador by P2)	<i>Do not challenge</i> ; <code>p.truth=0.20</code> , <code>thr=0.20</code>	Next challenge result is lose (claim false). Borderline equality missed a profitable challenge.
E3	<code>game_158 @ e7</code> (<i>Steal</i> , Captain by P3)	<i>Do not challenge</i> ; <code>p.truth=0.2503</code> , <code>thr=0.25</code>	Next challenge result is lose (claim false). Near-threshold score caused a miss.
E4	<code>game_417 @ e49</code> (<i>Foreign Aid</i> block, Duke by P4)	<i>Challenge</i> ; <code>p.truth=8.98e-34</code> , <code>thr=0.35</code>	Next challenge result is win (claim true). Block-claim handling is still calibration-sensitive.
E5	<code>dqn trace: game_106 @ t12</code>	DQN chose <i>Exchange</i> with 11 coins	Logged/legal action is <i>Coup</i> . In 10+ coin states, DQN chose non-Coup 151/181 times, so forced-Coup alignment is weak.

These cases suggest practical fixes: add threshold margin handling around equality edges, calibrate belief probabilities for repeated claims, and increase penalty shaping for non-Coup choices in 10+ coin states.

7 Challenges and Limitations

Partial observability and belief ambiguity. Coup is a partially observable game, so many states remain weakly identifiable from public actions alone. In early and symmetric phases, belief values can remain close across players (for example, near uniform levels), which limits confidence in single-step recommendations. This is a fundamental uncertainty challenge, not only an implementation issue.

Sequential interaction complexity. Core Coup interactions are multi-stage (action $\rightarrow$ block $\rightarrow$ challenge $\rightarrow$ reveal). Modeling these branches consistently in replay, simulation, and UI introduces engineering complexity and potential annotation noise. Small event-order errors can propagate into belief updates and advisor outputs.

Offline learning constraints. The current BC and DQN results are trained from logged simulator trajectories. This creates standard offline-RL limitations, including distribution shift and extrapolation error for unseen states. As a result, policy quality can degrade in rare but high-impact game situations.

Simplified opponent models. Opponent pools are represented by scripted bot styles (honest, bluffer, aggressive, cautious). These are useful for controlled experiments, but they do not fully capture adaptive human deception, long-horizon signaling, or meta-game adjustment across repeated matches.

Decision rule limitations. Challenge recommendations currently rely on thresholded posterior truth estimates with context adjustments. This improves transparency, but it remains myopic and does not explicitly optimize long-term game value (e.g., future retaliation risk, information signaling value, and endgame tempo control).

Evaluation scope. Current evaluation emphasizes simulation and offline metrics. Although this is reproducible, external validity is still limited without broader human-in-the-loop testing, cross-implementation comparisons, and larger stratified studies across player counts and table dynamics.

8 Conclusion

This project provides a reproducible Coup research pipeline focused on **round strategy optimization + challenge decision optimization**. The current results show a clear baseline: learned policies beat random, and BC leads DQN in this dataset regime. The updated rule layer (including forced Coup at 10+ coins) improves consistency between game regulation, advisor output, and replay/simulation data.

9 Future Work

1. **Belief calibration and uncertainty quantification.** Add post-hoc calibration (for example, isotonic or temperature-style calibration), reliability diagrams, and confidence-aware advisor outputs that explicitly communicate uncertainty bands.
2. **Thompson-sampling advisor policy.** Integrate Thompson Sampling for claim challenge and action selection using context-specific posteriors, then compare against fixed-threshold modes (Conservative/Balanced/Aggressive).
3. **Scenario benchmark suite.** Build a standardized scenario set covering Tax conflicts, Steal block trees, Assassinate–Contessa branches, forced-Coup states, and one-influence endgames for deterministic policy-level comparison.
4. **Online and hybrid training.** Extend from purely offline training to constrained self-play fine-tuning, while preserving rule-consistency checks (legality rate and forced-Coup compliance) during training.
5. **Opponent modeling and adaptation.** Learn opponent-type embeddings and dynamic response policies so advisor behavior adapts to table style rather than using only static global thresholds.

6. **Human-centered evaluation.** Run user studies on explanation quality, decision trust, and actionability, including evaluation of top- k conditional paths and warning language for novice and experienced players.
7. **Entropy-regularised policy learning.** Extend the Boltzmann softmax inference rule to a training-time objective by replacing the mean-squared Bellman loss with a soft Bellman target that includes an entropy bonus (a value-based analogue of soft actor-critic). This couples the exploration schedule to the learning signal rather than applying it only at action-selection time.
8. **Posterior-sampling belief queries.** When evaluating candidate actions, sample full role assignments from the belief distribution and score actions under each sample, rather than always conditioning on the posterior mean. This is the belief-layer analogue of Thompson Sampling and directly implements the “explore the path and the probabilities” idea at the layer where hidden-role uncertainty actually lives.

References

- [1] Leslie Pack Kaelbling, Michael L. Littman, and Anthony R. Cassandra. Planning and acting in partially observable stochastic domains. *Artificial Intelligence*, 101(1–2):99–134, 1998. doi:10.1016/S0004-3702(98)00023-X.
- [2] Christopher J. C. H. Watkins and Peter Dayan. Q-learning. *Machine Learning*, 8(3–4):279–292, 1992. doi:10.1007/BF00992698.
- [3] Leo Breiman. Random forests. *Machine Learning*, 45(1):5–32, 2001. doi:10.1023/A:1010933404324.
- [4] Stephane Ross, Geoffrey Gordon, and Drew Bagnell. A reduction of imitation learning and structured prediction to no-regret online learning. In *Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics (AISTATS)*, PMLR 15:627–635, 2011.
- [5] Takayuki Osa, Joni Pajarinen, Gerhard Neumann, J. Andrew Bagnell, Pieter Abbeel, and Jan Peters. An algorithmic perspective on imitation learning. *Foundations and Trends in Robotics*, 7(1–2):1–179, 2018. doi:10.1561/23000000053.
- [6] Volodymyr Mnih et al. Human-level control through deep reinforcement learning. *Nature*, 518:529–533, 2015. doi:10.1038/nature14236.
- [7] Long-Ji Lin. Self-improving reactive agents based on reinforcement learning, planning and teaching. *Machine Learning*, 8:293–321, 1992. doi:10.1007/BF00992699.
- [8] Sergey Levine, Aviral Kumar, George Tucker, and Justin Fu. Offline reinforcement learning: Tutorial, review, and perspectives on open problems. *arXiv preprint arXiv:2005.01643*, 2020. doi:10.48550/arXiv.2005.01643.

- [9] Tom Fawcett. An introduction to ROC analysis. *Pattern Recognition Letters*, 27(8):861–874, 2006. doi:10.1016/j.patrec.2005.10.010.
- [10] Alexandru Niculescu-Mizil and Rich Caruana. Predicting good probabilities with supervised learning. In *Proceedings of the 22nd International Conference on Machine Learning (ICML)*, pages 625–632, 2005. doi:10.1145/1102351.1102430.
- [11] Marina Sokolova and Guy Lapalme. A systematic analysis of performance measures for classification tasks. *Information Processing & Management*, 45(4):427–437, 2009. doi:10.1016/j.ipm.2009.03.002.